

PLAN4ARI

Deliverable D2.1

Updated Scientific Plan and First Running Mockup for Low-Code Programming with Generative and Deliberative AI

DRAFT - VERSION 0.11
Planning for Artificial Robotic Intelligence and Beyond

Project acronym	Plan4ARI
Project duration	48 months
Work package/activity	Activity 2 - Low-code programming exploiting generative and deliberative AI
Task basis	Task 2.1 - Update of the scientific plan with the latest state-of-the-art methodologies and tools
Activity / task period	Activity 2: M1–M36; Task 2.1: M1–M12 (proposed revised specification)
Contractual delivery point	M12 - scientific plan and first running mockup
Document status	Draft for consortium review - Version 0.11
Lead beneficiary	To be confirmed (technical drafting: CNR-STIIMA/Plan4ARI team)
Dissemination level	RESERVED
Evidence cut-off	Literature review: 21 July 2026; mockup and submission planning update: 23 September 2026

Document control

Version	Date	Author/reviewer	Change description
0.1	20 July 2026	Plan4ARI team	Initial outline
0.9	21 July 2026	Draft prepared for consortium review	Filled scientific plan; literature update; revised architecture and roadmap
0.10	23 September 2026	Draft for consortium review	M12 running mockup restored to D2.1 scope; detailed prototype description from [1]; journal submission planned by 23 October 2026
0.11	23 September 2026	Draft for consortium review	Added Section 9 on COMAU contributions and evidence records; linked requirements and mockup design; aligned T2.1 to M1–M12 in the proposed specification revision
1.0	TBC	TBC	Planned approved version - pending approval

Approval record

Role	Name	Date	Status
Technical lead	TBC	TBC	Pending
Host Institution representative	TBC	TBC	Pending
PI/Project Coordinator	TBC	TBC	Pending
Quality reviewer	TBC	TBC	Pending

Purpose and boundary of this deliverable

D2.1 comprises the updated scientific and technical plan for Activity 2 and the first running mockup of the low-code programming module, due at M12. The mockup is described in the A2 Research Project Proposal [1], particularly Section 2.1, Figure 1 and Section 3. This revision confirms its inclusion in the M12 delivery and specifies the demonstration and evidence to accompany it. Availability at M12 is a project commitment; this draft does not assert that the demonstration, experiments or journal submission have already been completed. Tasks 2.2–2.8 will extend, integrate and validate this initial mockup. The objectives and contractual KPIs of Activity 2 remain unchanged unless formally amended.

Drafting note. Dates, beneficiary ownership, dissemination level and formal approval information are intentionally left for consortium confirmation. Proposed technical decisions are identified as such and should be converted into approved decisions during the D2.1 review.

Contents

1	Executive summary	7
1.1	Headline update decisions	7
1.2	Expected impact on Activity 2	8
1.3	M12 mockup and near-term scientific output	8
1.4	COMAU industrial design contribution	8
2	Objectives, scope and original baseline	9
2.1	Activity 2 objective	9
2.2	Original five-step baseline	9
2.3	Contractual tasks and timing	9
2.4	Integration boundaries	9
2.5	Terminology corrections	10
2.6	Industrial platform baseline	10
3	Review methodology and evaluation criteria	11
3.1	Evidence collection	11
3.2	Analysis of available tools and platforms	11

4	Updated academic state of the art	15
4.1	From direct LLM planning to hybrid neuro-symbolic systems	15
4.2	Natural language to formal planning models	15
4.3	Planning and scheduling under resource and temporal constraints	15
4.4	Code generation, Code as Policies and symbolic code	15
4.5	Low-code/no-code industrial robot programming	16
4.6	Grounding symbolic plans to executable skills	16
4.7	Scene graphs and operational knowledge	16
4.8	Safety and verification trends	16
4.9	Evaluation scenarios and expected evidence	17
5	Gap analysis against the original Plan4ARI plan	18
5.1	Where the updated plan remains innovative	18
5.2	Developments that can be adopted rather than rebuilt	19
6	Updated scientific and technical architecture	20
6.1	Stage 1 - Controlled discovery	20
6.2	Stage 2 - Capability and skill registry	20
6.3	Stage 3 - Formalisation through a typed intermediate representation	22
6.4	Stage 4 - Mixed-initiative user dialogue	22
6.5	Stage 5 - Hybrid planning and scheduling	22
6.6	Stage 6 - Grounding to execution	22
6.7	Stage 7 - Verification and digital twin	22
6.8	Stage 8 - Execution, monitoring and replanning	22
6.9	Code-generation assurance gate	22
6.10	Security architecture	22
6.11	First running mockup at M12	23
6.12	Industrial input to mockup design	24
7	Method selection and decision rationale	25
7.1	Selected baseline stack	25
7.2	Planner-selection logic	25
7.3	Research hypotheses	25
7.4	Fallback strategy	25

8 Benchmarking and validation plan	27
8.1 Benchmark philosophy	27
8.2 Benchmark domains	27
8.3 Dataset structure	27
8.4 Metrics and proposed thresholds	27
8.5 Baselines and ablations	28
8.6 Statistical treatment	28
8.7 Mockup evaluation and evidence for the journal manuscript	28
9 COMAU contributions to industrial design	30
9.1 Reporting scope and evidence status	30
9.2 Potential contributions and expected evidence	30
9.3 Contribution records	30
9.4 Design decisions to document	30
9.5 Contribution summary for the reporting period	31
10 Updated implementation roadmap	32
10.1 M12 mockup and work breakdown M13–M36	32
10.2 Mapping to contractual tasks	32
10.3 Responsibilities	32
10.4 Decision gates	33
10.5 Scientific manuscript and submission target	33
11 Risks, ethics and regulatory alignment	34
11.1 Technical and operational risk register	34
11.2 Human oversight and explainability	34
11.3 EU AI Act	35
11.4 Machinery and cybersecurity regulation	35
11.5 Data protection and confidentiality	35
11.6 Research ethics	35
12 Conclusions and proposed updates	36
12.1 Final scientific position	36
12.2 Proposed approval decisions	36
12.3 Open research questions	36

Abbreviations and terms

Term	Definition
AAS	Asset Administration Shell.
ARI	Artificial Robotic Intelligence.
BT	Behavior Tree.
CP/CP-SAT	Constraint programming/satisfiability-based constraint programming.
CSTNU	Conditional Simple Temporal Network with Uncertainty.
DDS	Data Distribution Service, the communication middleware underlying common ROS 2 deployments.
Domain	Formal description of types, predicates/fluents and action schemas for a class of planning problems.
Grounding	Binding abstract symbols/actions to concrete objects, parameters, resources and executors.
IR	Intermediate representation.
LLM-modulo	Architecture in which an LLM interacts with external model-based solvers or verifiers.
MCTS	Monte Carlo Tree Search.
MI.RA/Dexter	COMAU model-driven, low-code industrial software platform used as Plan4ARI integration target.
No-code	Programming through goals and domain concepts without writing conventional source code.
OPC UA	Open Platform Communications Unified Architecture.
PDDL	Planning Domain Definition Language.
PDDL 2.1	PDDL extension for numeric fluents and durative actions.
PlanSys2	ROS 2 planning and execution framework.
ROS 2	Robot Operating System 2.
Scene graph	Graph representation of entities and their semantic, spatial or functional relations.
Skill	Reusable executable capability with explicit parameters, conditions, effects and interface.
STN/STNU	Simple Temporal Network/Simple Temporal Network with Uncertainty.
TAMP	Task and Motion Planning.
UP	Unified Planning library.
VAL	PDDL plan validation tool.
VLA	Vision-Language-Action model.
VLM	Vision-Language Model.

1 Executive summary

The original [Plan4ARI](#) Activity 2 concept remains well aligned with the central problem of industrial robotic programming: process experts should be able to describe desired outcomes without mastering robot languages, PLC configurations, planning formalisms, or the complete MI.RA/Dexter block library. The project baseline correctly anticipated that generative AI should be combined with deliberative AI rather than trusted as a stand-alone planner [2]. The scientific landscape between 2024 and July 2026, however, changes the most credible way to implement that vision.

The strongest finding of the review is a decisive move away from “LLM-alone” planning. Surveys and benchmark studies now distinguish planning quality through completeness, executability, optimality, representation, generalization, and efficiency, and continue to show that direct autoregressive plan generation is unreliable on complex state-tracking, resource, and constraint problems [3–8]. The dominant high-reliability pattern is hybrid: the language model interprets intent, extracts knowledge, proposes abstractions or repairs formal models; deterministic components compile, validate, solve, and execute the resulting planning problem [7, 9–15].

The same pattern is visible in industrial low-code/no-code robotics. Recent work uses LLMs as intent compilers that transform natural-language instructions or existing robot programs into structured process steps, low-code abstractions, or executable code. Accuracy improves when the output is constrained by a known function library, a standardized data model, a structured prompt, explicit rules, an intermediate representation, simulation, and human review [16–19]. Model size alone is not a dependable proxy for performance; the architecture and guardrails around the model are more important for industrial reliability.

Code as Policies is therefore related to low-code/no-code robot programming, but it should not be treated as the complete [Plan4ARI](#) architecture. It established that code-capable language models can synthesize robot policy programs that call perception and control APIs [20]. For [Plan4ARI](#), the safer interpretation is “code as a grounded, verified implementation artifact”: code generation may create adapters, skill executors, or orchestration logic, but formal planning, safety constraints, and execution authority remain outside the LLM. Generated code must pass static analysis, tests, digital twin validation, human approval, and signed deployment procedures before it reaches an industrial controller.

The proposed focus on discovering knowledge from the operational environment is scientifically sound and should serve as the differentiating focus of Activity 2. Scene graphs remain valuable for grounding objects, states, and relations, but they cover only part of the information needed for executable planning. The updated plan broadens the “knowledge base” into an evidence-fused capability and skill registry built from controlled ROS 2 graph introspection, PlanSys2 executors, and MI.RA/Dexter blocks, PLC/OPC UA interfaces, AutomationML or Asset Administration Shell models, code, manuals, process documentation, digital twins, and scene graphs. This addresses a major unresolved gap: translating a symbolic plan into behaviors that the current robotic cell can execute.

ROS 2/DDS discovery can expose nodes, topics, services, actions, parameters and message types, but those interfaces do not by themselves reveal semantic preconditions, effects, resources, timing, safety class or quality. These must be inferred conservatively from multiple sources, stored with provenance and confidence, and confirmed through tests or human review.

1.1 Headline update decisions

1. Retain the contractual objective: natural-language, low-code programming for process experts with deterministic and optimized task plans.
2. Broaden the research scope from human-robot interaction alone to operational knowledge discovery, capability modeling, neuro-symbolic planning, scheduling, skill grounding, execution, and runtime monitoring.
3. Replace direct natural-language-to-PDDL generation with natural language to a typed intermediate representation, followed by deterministic compilation to PDDL 2.1 and validator-in-the-loop repair.
4. Introduce a capability/skill registry as the central knowledge asset. It must represent interfaces, parameters, preconditions, effects, resources, durations, uncertainty, safety constraints, provenance, and test status.
5. Use a portfolio of classical, temporal, and constraint-based planners. The LLM proposes and repairs models; it is not the sole scheduler or optimizer.

6. Generate Behavior Trees from a selected and validated plan, not as a substitute for the planning search. Use STN/STNU mechanisms mainly for dispatch and temporal robustness.
7. Reposition the digital twin as a verification, timing-estimation, optimization, and code-test environment. MCTS remains an optional method for selecting stochastic or online problems, not the default global optimizer.
8. Ground every symbolic action to an existing trusted skill. Generate code only for missing glue or non-safety-critical executors through a gated software-assurance pipeline.
9. Execute and monitor through PlanSys2 and/or BehaviorTree.CPP adapters, with explicit state/effect validation and bounded replanning.
10. Keep model choice replaceable through a provider-neutral interface and benchmark model families at decision gates rather than freezing a rapidly aging model version.

1.2 Expected impact on Activity 2

The update does not change the intended 66% reduction in programming time, the use by non-pretrained workers, or the target scale of three mobile manipulators or four industrial arms [2]. It adds measurable intermediate outcomes: time to discover and onboard a cell, capability-registry accuracy, domain and problem validity, plan executability, action-grounding success, repair effort, optimality gap, runtime robustness, human-intervention rate and software-assurance evidence. These metrics make the project scientifically stronger and reduce the risk that a syntactically correct PDDL file is incorrectly interpreted as a successful no-code programming result.

The proposed restructuring is a better implementation of the original “**divide and conquer**” idea. The key innovation is the verified bridge between the live industrial system and a formal planning model, and back to executable skills. That bridge is currently less mature than natural-language interaction itself and offers a clearer research contribution.

1.3 M12 mockup and near-term scientific output

D2.1 will include the first running mockup at M12, alongside the scientific plan. Its reference implementation is the neuro-symbolic workflow described in [1]: operator interaction, structured problem generation, deterministic PDDL compilation, validation and repair, symbolic planning, and grounding to ROS 2/PlanSys2 execution. Section 6.11 defines the initial demonstration scope and distinguishes it from the broader research extensions.

A scientific manuscript presenting the mockup, its methodology and experimental evaluation is planned for submission within one month of this revision, with a target date of 23 October 2026. The candidate venues are IEEE Transactions on Automation Science and Engineering (IEEE T-ASE) or IEEE Transactions on Industrial Informatics (IEEE TII). One target journal will be selected. This is a planned submission, not an acceptance or publication claim. The calendar target is separate from project month M12.

1.4 COMAU industrial design contribution

Section 9 provides the reporting structure for COMAU contributions to requirements, architecture, use-case definition and mockup review. It distinguishes the pre-existing Dexter platform from project-specific design work and requires each contribution to be linked to a source, a decision and a mockup artefact. The contribution categories are candidates for confirmation; this revision does not certify their completion or assign personnel effort.

2 Objectives, scope and original baseline

2.1 Activity 2 objective

Activity 2 aims to decrease robot programming time and allow process experts, rather than robotic experts, to configure industrial robot applications through natural interaction. The modules are to be integrated into COMAU MI.RA/Dexter and the broader **Plan4ARI** controller architecture [2]. This deliverable updates the means, not the end objective.

2.2 Original five-step baseline

Table 2 Original five-step baseline of Activity 2 (Project Description).

Original step	Baseline intent	Original maturity/issue
A. Knowledge-base generation	Use generative AI to create a domain knowledge base from a process expert's natural-language description.	TRL 2 to 4; high dependence on prompt quality and human checking.
B. Natural language to PDDL	Use a lightweight LLM to generate PDDL coherent with the knowledge base and iteratively check it.	TRL 4 to 6; direct generation conflates semantic interpretation and formal syntax.
C. PDDL to Behavior Tree with time variance	Transform PDDL into a BT and incorporate duration uncertainty with STNU/CSTNU mechanisms.	TRL 2 to 4/4 to 6; original text treats BT search as a workaround for limited uncertain temporal planning.
D. Digital-twin optimization	Virtually execute alternative BT branches and use MCTS to identify an optimal policy.	TRL 2 to 4; potentially expensive and not always the correct optimisation formulation.
E. Execution and online replanning	Execute the selected BT, monitor context and use anytime plus heuristic replanning.	TRL 4 to 6; needs clearer safety, consistency and bounded-recovery rules.

2.3 Contractual tasks and timing

Table 3 Contractual tasks of Activity 2 and their periods.

Task	Contractual title/focus	Period
T2.1	Update of the scientific plan with the latest methodologies and tools	M1–M12 (proposed revision)
T2.2	PDDL 2 generation for robotic tasks with lightweight LLM from natural language	M13–M24
T2.3	PDDL 2 to Behavior Tree generation with time-variance modelling	M18–M24
T2.4	Task and motion planning optimisation in a digital-twin environment	M22–M30
T2.5	Optimal Behavior Tree execution and online replanning	M28–M36
T2.6	Integration inside MI.RA/Dexter low-code	M28–M36
T2.7	Continuous integration	M13–M36
T2.8	Benchmarking and evaluation	M7–M12 and continuing support

2.4 Integration boundaries

- **Open IPC:** non-hard-real-time cognitive programming, ROS 2 integration, perception, task and motion planning, external interfaces, and MI.RA/Dexter integration.

- **Core IPC:** hard-real-time motion control and safety-critical control functions. Activity 2 must not allow generated code or an LLM to bypass this boundary.
- **MI.RA/Dexter:** dispatcher, execution monitor, and model-driven low-code environment; source of available blocks and target for deployed workflows.
- **ROS 2/PlanSys2/BehaviorTree.CPP:** candidate open integration layer for symbolic planning, action execution, and reactive orchestration.

2.5 Terminology corrections

The original documents use “PDDL2”. The updated plan uses “PDDL 2.1” when referring to durative actions, numeric fluents, and temporal planning [21]. Where trajectory constraints, preferences, HTN structure or stochastic outcomes are required, a compatible extension or a complementary formalism may be needed. PlanSys2 compatibility must be checked against the exact PDDL subset used in each implementation.

The term “knowledge base” is retained at the project level but decomposed technically into: (i) a capability and skill registry; (ii) a world/scene state; (iii) process and safety constraints; (iv) planning domains and problems; and (v) execution history and evidence. This avoids using one undifferentiated repository for both static capability knowledge and rapidly changing scene state.

2.6 Industrial platform baseline

According to the project team, MI.RA/Dexter currently provides a block-based IDE using Model-Driven Engineering. Programmers compose applications by connecting modules from a library and may develop additional modules. This existing environment is the industrial starting point for Activity 2. The research concerns how task intent, capability semantics and planning can support workflow generation. Availability of the existing platform does not, by itself, establish a project-specific contribution during the reporting period. COMAU should confirm the platform version, representative modules and applicable interface documentation.

3 Review methodology and evaluation criteria

3.1 Evidence collection

The update is a targeted, decision-oriented literature and tool review on 21 July 2026. It focuses on the analysis of an eight-paper source list supplied for D2.1, the [Plan4ARI](#) project description [2], the updated D2.1 outline [22], the restructuring proposal [1], peer-reviewed papers from automated planning, robotics, manufacturing, and NLP, relevant preprints where the field is too recent for archival publication, and official project/tool documentation. Foundational work before 2024 is included when it defines the conceptual or software baseline.

The papers have been selected because of their impact in the fields of:

- LLM task planning and benchmarks; LLM-to-PDDL/domain generation;
- neuro-symbolic and LLM-modulo planning; task allocation and constraint scheduling; task-and-motion planning with VLMs;
- industrial natural-language robot programming; low-code robot programming;
- Code as Policies and code generation;
- ROS 2 planning/execution frameworks;
- skill and capability ontologies;
- OPC UA, AutomationML, and Asset Administration Shell capability models; scene graphs; temporal uncertainty;
- Behaviour Trees;
- regulatory requirements.

3.2 Analysis of available tools and platforms

3.2.1 Generative-model layer

The architecture should support multiple hosted and local models behind a common interface. Model versions will change faster than Activity 2 can be implemented. The selection therefore has to be benchmark-driven and revisited at M15, M21 and M27. Representative categories are: hosted frontier reasoning/coding models; open-weight 7–14B models for local structured extraction and repair; larger open-weight models for offline development; multimodal models for documentation and scene evidence; and small fine-tuned models for constrained command interpretation.

3.2.2 Planning, scheduling and validation tools

3.2.3 Tool and method comparison matrices

3.2.3.1 LLM planning patterns

3.2.3.2 Capability registry design comparison

3.2.4 Execution and robotics integration

3.2.5 Industrial semantics and engineering data

The discovery layer should be deliberately multi-protocol. ROS 2 is central to research and Open IPC integration, but industrial cells also expose information through PLC variables, OPC UA servers, device descriptions, engineering projects, and digital-twin artefacts. Capability-based robot frameworks, OPC UA skill models, AutomationML and Asset Administration Shell research all show that semantic descriptions are required to make functions discoverable and composable [28–33].

Table 4 Generative-model categories and their role in Plan4ARI.

Model category	Strengths	Limitations	Plan4ARI role
Hosted frontier model	Strong language/code reasoning, tool use, long context.	Data governance, cost, version drift, network dependency.	Reference benchmark; difficult cases; never sole safety authority.
Local open-weight 7–14B	Edge deployment, privacy, reproducibility, low marginal cost.	Lower reliability on complex formalisation; hardware constraints.	Primary candidate for structured extraction, classification and repair with validators.
Local larger open weight	Better reasoning and code generation while preserving data control.	GPU footprint, energy, integration complexity.	Offline domain induction and code assistance; optional on-premises service.
Multimodal/VLM	Can interpret diagrams, screenshots, manuals and scene observations.	Grounding and geometry remain uncertain; larger attack surface.	Evidence extraction and scene/context interpretation; outputs converted to explicit facts.
Fine-tuned compact model	Stable schema-specific output and predictable latency.	Dataset creation and maintenance; narrow generalisation.	Command frames, capability classification and high-volume repair tasks.

Table 5 Planning, scheduling and validation tools.

Tool/class	Function	Recommended use
Unified Planning	Python modelling and engine abstraction across multiple planning technologies [23].	Canonical programmatic planning API and planner portfolio integration.
Fast Downward family	Classical planning and strong heuristics.	Baseline for deterministic non-temporal subproblems.
POPF/OPTIC	Temporal planning, preferences and continuous/time-dependent costs.	Temporal candidate engines, subject to feature and licensing checks.
VAL	PDDL plan validation and diagnostics [24].	Independent plan validation and repair feedback.
Constraint programming/CP-SAT	Resource allocation, scheduling, precedence, time windows and optimisation.	Primary complement to PDDL for heterogeneous multi-agent scheduling [14].
STN/STNU/CSTNU tools	Temporal consistency, dispatchability and uncontrollable duration reasoning.	Execution-time temporal network and robustness layer, not a substitute for symbolic planning.
OMT/SMT solvers	Constraint satisfaction, temporal optimisation and verification.	Research option for hard multi-constraint problems and formal checks [13, 25].

Table 6 Comparison of LLM planning patterns.

Pattern	Advantages	Limitations	Plan4ARI status
Direct LLM plan	Simple interface and rapid prototype.	No guarantees; state/resource errors; difficult validation.	Baseline only.
LLM → PDDL problem → planner	Correct/optimal plan once the model is correct.	Depends on the existing domain and the correct translation.	Retain with IR and validators.
LLM domain + problem generation	Reduces knowledge-engineering effort.	Semantic domain errors and incomplete effects.	Research focus with evidence grounding.
LLM-modulo validator loop	External checks and structured repair.	More components and latency.	Primary architecture.
LLM + CP scheduling	Natural task decomposition plus formal resource scheduling.	The interface between task semantics and CP must be engineered.	Primary for multi-agent scheduling.
Code as policy	Expressive API composition and reactive logic.	Generated code risk and weak formal task guarantees.	Adapters/orchestration under assurance gate.
VLM/VLA end-to-end	Strong perception-action integration and generalization potential.	Data, certification, repeatability and control challenges.	Monitor; not Activity 2 baseline.
Scene-graph + planner	Explicit state grounding and inspectable relations.	Scene graph accuracy and domain adaptation.	Primary perception-to-symbolic route.

Table 7 Comparison of capability registry designs.

Approach	Strength	Weakness	Decision
ROS 2 graph only	Immediate runtime discovery and interface truth.	Little semantic meaning; only active nodes.	Use as one evidence source.
Documentation/RAG only	Rich semantics and process constraints.	May be stale and inconsistent with deployment.	Use with provenance and runtime confirmation.
Scene graph only	Strong physical state and object relations.	No software/device capability semantics.	Use for world-state branch.
Skill library annotations	High-quality executable semantics.	Manual authoring and maintenance burden.	Authoritative when approved.
Ontology/AAS/AutomationML	Interoperability and machine-readable engineering context.	Coverage and vendor adoption vary.	Adopt compatible mappings.
Evidence-fused registry	Combines runtime truth, semantics and state.	Research complexity and conflict resolution.	Selected Plan4ARI approach.

Table 8 Execution and robotics integration components.

Component	Current capability	Plan4ARI decision
PlanSys2	PDDL-based planning system for ROS 2 with plan execution and Behavior Tree mechanisms [26].	Preferred reference executor and integration target; verify supported PDDL features and lifecycle behaviour.
BehaviorTree.CPP/ BehaviorTree.ROS2	Production-oriented reactive BT framework with ROS 2 wrappers.	Preferred BT runtime for generated execution structures and MI.RA/Dexter adapters.
SkiROS2	Skill-based control, semantic world model, pre/hold/postconditions and automatic planning-domain generation [27].	Important comparator and source of design patterns for the capability registry.
ROS 2 graph/introspection APIs	Discover nodes, topics, services, actions, parameters, types and QoS.	Use through authenticated, least-privilege tools; never equate interface discovery with semantic capability knowledge.
MoveIt 2/TAMP components	Motion planning and manipulation integration.	Feasibility checks and skill implementation; Activity 2 should exchange symbolic goals and constraints with Activity 4.
MI.RA/Dexter	COMAU model-driven low-code blocks, digital representation and dispatcher/execution-monitor role.	Industrial integration target and authoritative source of approved executable blocks.

Table 9 Discoverable information per source and remaining gaps.

Source	What can be discovered	What remains missing
ROS 2 graph	Runtime topology, interfaces, message/action/service types, parameters, active executors.	Business meaning, preconditions/effects, safe operating envelope, quality and resource constraints.
MI.RA/Dexter block metadata	Approved blocks, ports, configuration parameters, workflow compatibility.	Formal planning semantics unless explicitly annotated.
OPC UA/companion models	Assets, variables, methods, states, device information and selected skill interfaces.	Uniform semantics across vendors and complete action models.
AutomationML/AAS	Engineering structure, topology, capabilities, data sheets and lifecycle information.	Runtime state and direct executability unless linked to services/skills.
Code and manifests	Actual API calls, dependencies, types, tests and package metadata.	Intent, process constraints and safe use conditions may be implicit.
Manuals and process documents	Natural-language constraints, setup procedures, tools and failure handling.	Machine-readable consistency and evidence of current deployment.
Scene graph/perception	Objects, attributes, states and spatial/functional relations.	Latent device capabilities and automation interfaces.

4 Updated academic state of the art

4.1 From direct LLM planning to hybrid neuro-symbolic systems

The literature no longer supports the use of a general-purpose LLM as the sole planner for industrial tasks. Plan-Bench and subsequent evaluations show persistent difficulty with exact state transitions, long horizons, resource accounting, and constraint compliance, even when reasoning-oriented models improve average performance [6–8]. Surveys now organise the field around external-module augmentation, search, fine-tuning, and planning formalisation rather than assuming that prompt engineering alone will produce dependable plans [3–6].

The LLM-modulo view provides the most useful conceptual basis for **Plan4ARI**: an LLM is an approximate knowledge source and semantic interface, while external model-based components provide checks, search and guarantees [7]. This is stronger than a one-way pipeline because solver and validator diagnostics are fed back into controlled repair. The design also makes uncertainty visible: when the system cannot ground a request or generate a valid model, it returns a structured failure or asks the user, rather than inventing an executable action.

4.2 Natural language to formal planning models

LLM+P demonstrated the core hybrid pattern: translate natural-language planning problems into PDDL, solve with a classical planner, and translate the solution back [9]. NL2Plan and later work decomposed PDDL creation into inspectable intermediate steps, reporting failure rather than always returning an invalid plan [10]. Domain-generation studies show moderate capability, not production-ready correctness, when LLMs infer action schemas from text [11]. More recent experiments indicate that solver/validator feedback and iterative revision can more than double the success of smaller open models, while apparently intuitive grammar constraints do not automatically improve semantic quality [12].

The implication is that **Plan4ARI** should not ask a model to write the final PDDL in one step. A typed JSON or ontology-backed intermediate representation should capture objects, facts, goals, action candidates, assumptions, and unknowns. A deterministic compiler then produces PDDL. Validation reports constrain the repair model to the existing domain signature. Domain generation is treated as model learning with tests, not as text generation.

4.3 Planning and scheduling under resource and temporal constraints

Recent hybrid multi-agent work uses an LLM for task decomposition and constraint programming for allocation and scheduling under resource and time limits [14]. This division is particularly appropriate for **Plan4ARI**'s target fleets. PDDL expresses state-changing actions and durative structure well, while CP/CP-SAT is often more direct for cumulative resources, assignment, calendars, time windows and makespan optimisation. A planner portfolio can select or compose these methods rather than forcing all problems into one formalism.

Temporal uncertainty also needs a layered treatment. PDDL 2.1 can represent durative actions, but uncontrollable durations and robust execution require additional temporal-network or stochastic mechanisms [21, 34]. STNU/CSTNU techniques are best positioned after plan generation to test dynamic controllability, create dispatchable windows and monitor temporal deviations. They do not by themselves discover actions, solve symbolic goal ordering or replace task allocation.

4.4 Code generation, Code as Policies and symbolic code

Code as Policies showed that code-trained LLMs can synthesise robot policy programs that call APIs, implement feedback loops and perform spatial calculations [20]. It is an enabling framework for low-code/no-code programming because it turns language into executable composition. Code-as-Symbolic-Planner moves the same coding ability into the planning layer, generating solver/checker code for constrained TAMP and reporting an average improvement in success over the direct baseline across diverse tasks [15]. Formal-verification work similarly compiles natural-language constraints into satisfiability problems solved by sound external tools [13].

These results support code generation as a representation and integration mechanism, but not unrestricted autonomous deployment. Industrial robots require stable APIs, deterministic bounds, cybersecurity review and configuration management. **Plan4ARI** should distinguish: (i) symbolic code executed in a sandbox to solve or check a planning problem; (ii) orchestration code that calls approved skills; (iii) new skill implementations; and (iv) safety-critical motion/control code. Only the first two are suitable for routine automated generation. The third requires a full software-assurance gate. The fourth is outside the scope of Activity 2, generative AI.

4.5 Low-code/no-code industrial robot programming

Industrial studies from 2024–2026 confirm that LLMs can lower the abstraction barrier. LLMs have been used to transform conventional robot programs into low-code abstractions [16], convert natural-language production instructions into structured robot commands with rule-based constraints and reviewable intermediate artefacts [17], and generate C++ from process steps through a high-level function library integrated with ROS 2, MoveIt 2 and AutomationML [18]. ROS-LLM provides a broader open architecture with sequences, Behaviour Trees, state machines and feedback [19].

The consistent lesson is that accessibility comes from a curated skill library and structured workflow, not from exposing raw vendor commands to a chatbot. The model should choose and parameterise approved functions; the runtime should enforce types, resource constraints and safety. Human review is most valuable at semantic ambiguity, new domain creation and first deployment, not as a substitute for machine validation on every syntax error.

4.6 Grounding symbolic plans to executable skills

Symbolic validity is not robotic executability. A plan action, such as `pick(object, location)`, is usable only if the current cell exposes an executor with compatible arguments, required perception, grasping, motion and resource conditions. PlanSys2 provides a ROS 2 planning/execution framework, while SkiROS2 demonstrates a skill model with conditions, semantic world knowledge and automatic planning-domain generation [23, 26, 27]. Capability surveys and ontologies emphasise the need for standardised descriptions that distinguish advertised capability from operational capability and bind abstract functions to concrete implementations [28–33].

This is the most important addition to the original **Plan4ARI** plan. The activity should treat the grounding map as a first-class artefact: each formal action schema is linked to one or more executable skills, interfaces and evidence sources. Grounding includes not only matching names but checking types, resources, current availability, parameter ranges, quality, timing, safety conditions and observability of effects.

4.7 Scene graphs and operational knowledge

Scene graphs remain a strong structured representation between perception and planning. SayPlan uses 3D scene graphs to scale language-based task planning [35]. Domain-conditioned scene graphs map domain-specific scene relations directly to symbolic planning predicates and improve state estimation and planning success over less structured multimodal baselines [36]. VLM-TAMP uses a vision-language model to propose semantically useful subgoals while a TAMP system preserves geometric feasibility [37]. Safe Planner adds a separate safety prediction module to guide high-level planning [38].

For **Plan4ARI**, scene graphs should represent mechanical parts, tools, fixtures, accessibility, support, contact, fastening and precedence relations as proposed in the restructuring document [1]. They should also serve as runtime execution evidence. Nevertheless, the scene graph is only the physical-state branch of a broader operational knowledge graph that includes devices, software agents, skills, resources and process constraints.

4.8 Safety and verification trends

Recent safety-oriented LLM planning work adds dedicated safety modules, formal logic, temporal logic or control-barrier components rather than relying on the same model to generate and self-check a plan [38–40]. This supports

an architectural rule for **Plan4ARI**: a probabilistic generator must not be its own only validator. Independent deterministic checks, policy gates, and a human approval path are required for new capabilities, code, and safety-relevant constraints.

4.9 Evaluation scenarios and expected evidence

Table 10 Evaluation scenarios and expected evidence.

ID	Scenario	Input variations	Expected evidence
C1	Pick a red object from bin A and place it in bin B.	Object absent, multiple red objects, gripper unavailable.	Clarification or discovery, valid problem, grounded plan, effect check.
C2	Inspect components and rework failures.	Unknown inspection service, conditional result, limited station capacity.	Information-gathering model, conditional/replanning behaviour.
C3	Remove cover after fasteners.	Different fastener count, occluded screw, wrong tool.	Scene-graph precedence, tool capability, plan repair.
C4	Weld seams with one robot and one human.	Human delay, shared zone, no mid-seam interruption.	Temporal schedule, safety constraints, stop/resume policy.
C5	Allocate transport tasks to three mobile manipulators.	Battery/resource limits, blocked route, heterogeneous payload.	CP allocation, PDDL state update, makespan comparison.
C6	Generate a missing ROS 2 adapter for an approved inspection API.	Incorrect type, dependency issue, simulated failure.	Code gates, tests, signed artefact or rejected release.
C7	Onboard a new cell from ROS 2 + AutomationML + manuals.	Conflicting documentation and inactive node.	Registry provenance, conflict resolution, domain tests.
C8	Runtime tool failure during assembly.	Alternative tool exists/no alternative.	Effect detection, grounded replan or safe stop.

5 Gap analysis against the original Plan4ARI plan

Table 11 Original baseline assumptions versus current evidence and updated treatment.

Baseline assumption	Current evidence	Decision	Updated treatment
LLM creates the knowledge base from natural language.	Text-only domain generation is moderately successful but remains under-specified; live capabilities and interfaces are usually absent.	REFINE	Build an evidence-fused capability/skill registry from runtime, engineering and perceptual sources; use LLM extraction with provenance.
Lightweight LLM directly generates PDDL.	Structured pipelines, intermediate representations and validator feedback outperform one-shot generation.	REFINE	NL → typed IR → deterministic compiler → validators → constrained repair.
Multi-agent LLM/RL is the main reliability mechanism.	Multiple agents can improve workflow decomposition but do not create guarantees and may amplify cost/inconsistency.	REPLACE	Use typed workflow nodes; deterministic validators and solvers are mandatory. Multi-agent roles are optional.
PDDL-to-BT with time variance can avoid temporal planners.	Temporal planners, OMT and CP tools are available; STNU is useful for robust dispatch, not a complete task planner.	REPLACE/REFINE	Plan with a portfolio; transform the selected plan to BT; use STN/STNU for execution windows and uncertainty.
BT contains all possible task plans and MCTS selects the optimum.	BTs encode execution policy; exhaustive alternative generation scales poorly and duplicates planner search.	REPLACE	Use planners/CP for search and optimisation. Use digital twin and MCTS selectively for uncertain online decisions or parameter tuning.
A heuristic emergency replanner restarts movement.	Unbounded heuristic recovery can violate assumptions and safety constraints.	REFINE	Use verified plan repair, pre-approved recovery BTs and safe stop when no validated recovery exists.
Scene understanding is mainly object registration/segmentation.	Recent work uses explicit scene graphs and domain-conditioned predicates, but executable planning also needs device and software capability knowledge.	REFINE	Combine scene graph with operational capability graph and execution registry.
Human-robot interaction is the principal field.	HRI remains important, but current innovation spans formalisation, scheduling, capability discovery, skill grounding and software assurance.	BROADEN	Keep dialogue and human oversight while making operational knowledge discovery the central research contribution.
Code generation can directly close the loop.	Code generation is powerful but unsafe without sandboxing, testing and deployment controls.	REJECT FOR DIRECT DEPLOYMENT	Generate only symbolic solver code, adapters and candidate executors; enforce full assurance gate.
Specific LLM versions can be selected early.	Model landscape changes faster than the project plan.	MONITOR	Benchmark replaceable model families and maintain an abstraction layer.

5.1 Where the updated plan remains innovative

- Automatic, evidence-backed discovery of the capabilities of a running ROS 2/industrial automation system and conversion into a formal planning model.

- Joint representation of advertised and operational capability, including provenance, confidence, temporal uncertainty and safety classification.
- Closed-loop generation, validation, repair and grounding of planning domains/problems against the actual robotic cell.
- Planner-agnostic combination of PDDL state reasoning, constraint scheduling, temporal dispatch and robot skill execution.
- Industrial code-generation governance that treats generated code as a controlled engineering artefact rather than an immediately executable policy.
- Use of scene graphs for mechanical assembly/disassembly reasoning inside a broader capability-aware neuro-symbolic architecture.

5.2 Developments that can be adopted rather than rebuilt

- Unified Planning for a common modelling/engine interface.
- VAL and planner diagnostics for plan validation.
- PlanSys2 and BehaviorTree.CPP/ROS2 for planning/execution integration.
- SkiROS2 design patterns for semantic skills and domain generation.
- Established ROS 2 introspection and lifecycle tooling.
- Existing PDDL, temporal, and CP solvers selected through feature tests.
- Standard container, CI, software-bill-of-materials and static-analysis tooling.

6 Updated scientific and technical architecture

The updated architecture maintains the original divide-and-conquer philosophy while changing the division points. The system is organised around an operational knowledge loop: discover what exists; normalise it as explicit capabilities; formalise the user's goal and the cell state; solve a deterministic planning/scheduling problem; ground every action to an executor; verify; execute; and update the knowledge from observed outcomes.

Updated Plan4ARI Activity 2 architecture

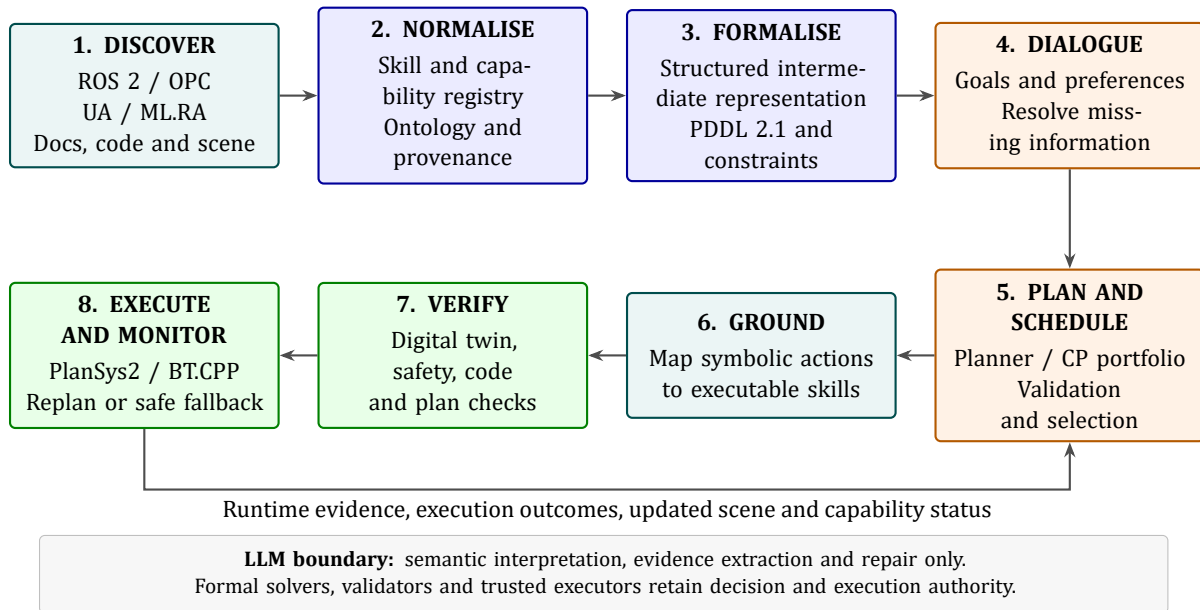


Figure 1 Updated end-to-end architecture for Activity 2, from controlled discovery to monitored execution.

6.1 Stage 1 - Controlled discovery

Discovery is performed through authenticated tools with explicit read permissions. ROS 2 discovery captures the computational graph and message/service/action definitions. PlanSys2 and ML.RA/Dexter adapters enumerate registered action executors and low-code blocks. Industrial adapters inspect authorized OPC UA methods and variables, PLC metadata, AutomationML/AAS models, and configuration files. Retrieval tools index manuals, process specifications, package manifests, code, and test evidence. Perception provides the current scene graph and object state.

The output is evidence, not yet a capability. Each observation is timestamped and linked to its source. Conflicts are retained for resolution. The system must distinguish “present on the network”, “callable”, “tested”, “approved for planning”, and “approved for execution”.

6.2 Stage 2 - Capability and skill registry

The registry is the stable semantic layer between the operational system and planning. A minimum skill record contains: unique identifier and version; provider and deployment endpoint; semantic description; typed inputs and outputs; preconditions, invariants and effects; required resources; interface binding; nominal and observed duration distributions; cost/quality models; safety class; failure modes and recovery options; effect-observation method; provenance and confidence; test status; and approval state.

Capability discovery extends beyond network sniffing

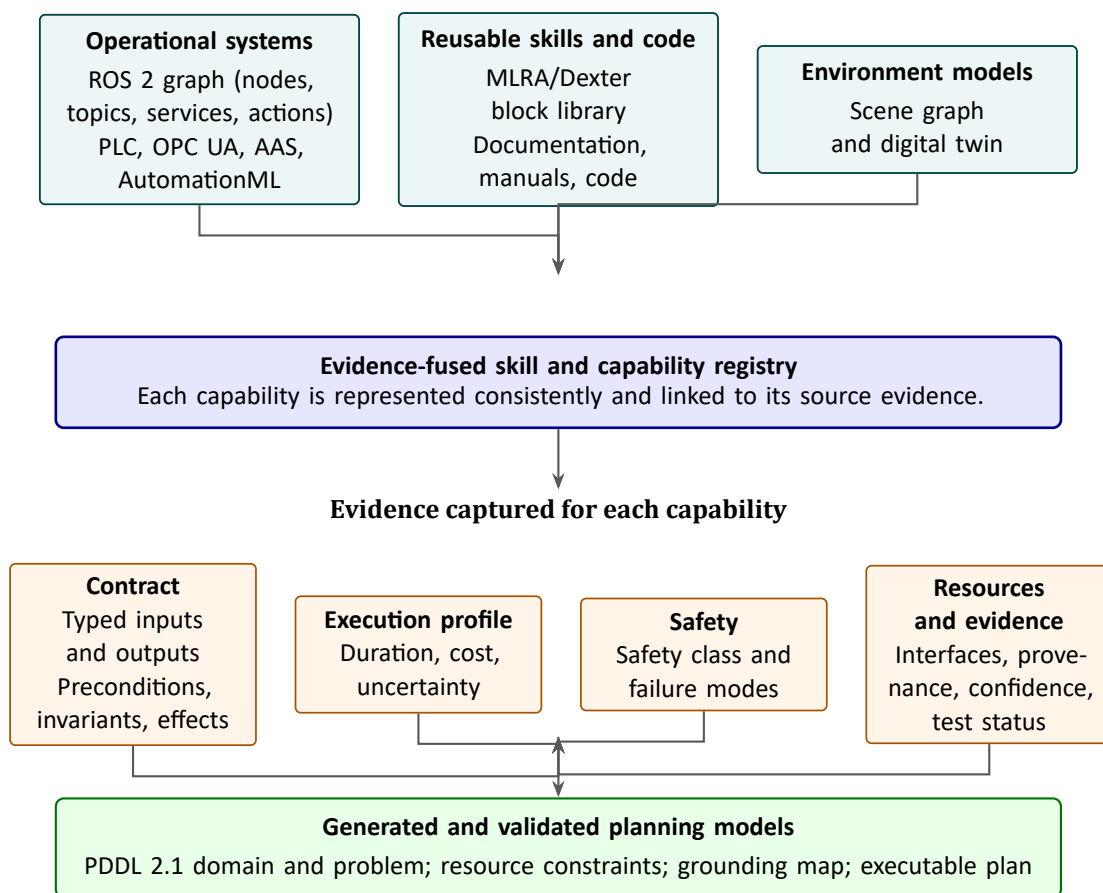


Figure 2 Multi-source discovery and the evidence-fused capability/skill registry.

6.3 Stage 3 - Formalisation through a typed intermediate representation

The language model does not write final PDDL as its first artefact. It populates a schema-constrained intermediate representation using only types, predicates, resources and skills available from the registry, while clearly listing assumptions and missing information. A deterministic compiler converts the IR into PDDL 2.1, PDDL constraints, a CP model or another selected formalism. This separation simplifies validation, testing, migration and model comparison.

6.4 Stage 4 - Mixed-initiative user dialogue

The dialogue component elicits goals, preferences, deadlines, quality criteria and acceptable trade-offs. It asks for human input only when the missing value cannot be derived from trusted sources or when alternatives have materially different safety, cost or process meaning. The user is shown a concise explanation of the interpreted goal, assumptions, unavailable capabilities and selected objective before planning.

6.5 Stage 5 - Hybrid planning and scheduling

A feature analyser selects the planning route. Classical state-transition problems use a PDDL planner. Durative and preference-rich problems use temporal planners. Multi-agent allocation and resource scheduling use CP/CP-SAT, optionally connected to a symbolic plan skeleton. TAMP or MoveIt 2 feasibility checks refine actions that require geometric validity. A portfolio returns candidate plans, diagnostics and objective values. The plan selector uses deterministic criteria; an LLM may explain trade-offs but does not override hard constraints.

6.6 Stage 6 - Grounding to execution

For each grounded planning action, the grounding service resolves an approved skill implementation and binds symbolic arguments to ROS 2 entities, MIRA/Dexter parameters, tools, objects and resources. It checks current availability, interface compatibility, parameter ranges and required observations. If a suitable executor is missing, the system returns a capability gap. A code-generation workflow may propose an adapter or executor only when the necessary lower-level functions are already approved and the generated code is not safety critical.

6.7 Stage 7 - Verification and digital twin

Before release, the system validates the domain/problem, planner output, and grounding map; checks temporal/resource consistency; simulates the workflow in the digital twin; tests generated adapters; estimates execution-time distributions; and verifies policy and safety constraints. Failed checks generate machine-readable diagnostics that are routed to the appropriate repair node. A repaired artefact must pass the complete validation chain again.

6.8 Stage 8 - Execution, monitoring and replanning

The approved plan is transformed into the execution structure supported by PlanSys2, BehaviorTree.CPP and MIRA/Dexter. Runtime monitoring compares expected and observed action effects, scene state, resource state and time windows. Small deviations are handled by a dispatch policy or an approved local recovery. A symbolic discrepancy triggers plan repair or replanning from the updated state. If no validated recovery exists, the system enters a safe state and requests human intervention.

6.9 Code-generation assurance gate

6.10 Security architecture

- Separate the LLM/tool network from safety-related control networks through the existing Open IPC/Core IPC boundary.

Table 12 Code-generation assurance gates.

Gate	Required evidence
G0 - Proposal	Generated source, declared purpose, target interface and provenance.
G1 - Static checks	Compilation, type checks, linting, security scanning, dependency allow-list and licence scan.
G2 - Unit tests	Generated tests plus project-maintained interface and boundary tests.
G3 - Simulation	Digital-twin execution, expected effects, timing limits and failure injection.
G4 - Integration	ROS 2/MI.RA/Dexter integration tests in an isolated test namespace.
G5 - Review	Human engineering approval, traceability to requirement and change record.
G6 - Release	Signed container/package, SBOM, version pinning and rollback plan.
G7 - Operational approval	Explicit approval for the target cell; generated code cannot migrate automatically between cells.

- Use least-privilege service accounts and allow-listed discovery actions.
- Treat documentation, ROS 2 messages and user prompts as untrusted input; defend against prompt injection and malicious metadata.
- Never expose arbitrary shell, package installation or network access to the model in the production path.
- Log prompts, tool calls, retrieved evidence, model/version, artefact hashes, validations and approvals.
- Sign deployable artefacts and support deterministic rollback.

6.11 First running mockup at M12

The M12 mockup implements the initial neuro-symbolic workflow described in [1, Sec. 2.1, Fig. 1 and Sec. 3]. It starts from an available PDDL domain, a natural-language operator request, optional textual context and access to the running ROS 2 system. The expected outputs are a validated problem, a selected symbolic plan and an execution-grounding map that connects planned actions to concrete PlanSys2/ROS 2 behaviours. The demonstration will use the simplified single-robot pick-and-place scenario identified in [1].

Operator interface and workflow. The graphical interface described in [1, Fig. 4] accepts task requests and optional context and presents execution feedback. The LangChain-based workflow maintains structured state containing the domain signature, request, context, ROS 2 evidence, generated problem, validation diagnostics, planner outputs and grounding information. Deterministic nodes perform parsing, compilation, validation and planner invocation. LLM engines generate or repair structured content; tool-using LLM agents query the robotic system when required.

Problem generation and repair. The workflow extracts the domain types, predicates, numeric fluents and action schemas. It resolves missing information through available ROS 2 introspection tools or operator clarification. The problem generator produces JSON containing objects, initial facts, numeric values, goals, assumptions and unresolved information. A deterministic compiler generates PDDL. Unified Planning model checks and VAL-based plan validation, as applicable to the selected planning features, provide diagnostics to a bounded repair loop. The demonstration record will identify the checks actually implemented.

Planning and execution grounding. A planner portfolio, accessed through Unified Planning and compatible engines such as POPF or OPTIC, generates candidate plans. The mockup will record the configured engines and the plan-selection criterion. Each selected action is matched to an available PlanSys2 executor or ROS 2 skill; concrete arguments and missing execution information are retrieved before dispatch. The run log will distinguish formal plan validity from successful skill binding and observed execution.

Missing executors and controlled code generation. The proposal describes an external code-generation service, accessed through a web-server interface, for synthesising a missing PlanSys2-compatible executor from available ROS 2 interfaces. Where demonstrated, this branch will record the generated artefact and the checks performed before execution, in line with Section 6.9. Unsupported actions must be reported as capability gaps. The

demonstration report will identify enabled branches and limitations rather than imply complete automatic skill synthesis.

Research extensions and maturity. Research Goal 2 in [1] relaxes the available-domain assumption through ontology-based skill discovery and domain generation. Research Goal 3 studies scene graphs for mechanical assemblies, disassembly BT generation and execution monitoring. These directions belong to the wider A2 programme. Any prototype elements shown at M12 will be labelled by their actual implementation and validation status. Full multi-source discovery, multi-robot evaluation, temporal robustness and industrial MI.RA/Dexter integration remain subsequent development objectives.

M12 evidence package. D2.1 will identify the versioned mockup release and execution environment, provide setup and demonstration instructions, record the input domain and operator requests, and retain generated JSON/PDDL, validation diagnostics, selected plans, action bindings and execution logs. A demonstration recording or live demonstration will accompany the technical description. Quantitative results will be added only after the corresponding experiments have been performed.

6.12 Industrial input to mockup design

The M12 mockup description should identify which requirements, constraints, examples or review findings originated from COMAU and how they affected the implementation. Section 9 provides the contribution matrix and evidence record for this attribution. Scientific method development, prototype implementation, industrial review and later MI.RA/Dexter integration must be recorded as distinct contributions. No first-year Dexter integration is claimed merely because the mockup is designed for subsequent integration.

7 Method selection and decision rationale

7.1 Selected baseline stack

Table 13 Selected baseline stack per layer.

Layer	Primary selection	Fallback/comparator	Rationale
Knowledge representation	Ontology-backed skill registry plus relational world/scene state.	Relational database or knowledge graph with equivalent schema.	Planning must be grounded in executable, versioned capabilities rather than free text.
LLM workflow	Provider-neutral tool-using workflow with typed state and schema outputs.	Single-model pipeline for baseline comparison.	Reduces coupling and allows deterministic nodes around probabilistic calls.
Formalisation	Typed JSON/ontology IR → deterministic PDDL 2.1 compiler.	Direct PDDL generation as experimental baseline.	Separates semantics from syntax and enables robust validation.
Planning	Unified Planning portfolio with classical and temporal engines.	Direct engine integration.	Supports feature-based engine selection and reproducible comparison.
Scheduling	CP/CP-SAT for resources, allocation and time windows.	PDDL temporal planner when model fits.	Better fit for heterogeneous resources and industrial scheduling.
Validation	VAL + UP model checks + planner diagnostics.	Additional SMT/OMT checks.	Independent deterministic evidence is required.
Execution	PlanSys2 + BehaviorTree.CPP/ROS2 adapters.	MI.RA/Dexter-native workflow executor.	Open research stack and industrial target can be compared and integrated.
Temporal uncertainty	STN/STNU-based dispatch and monitoring.	Robust margins or stochastic planner for specific cases.	Keeps uncertainty handling explicit at execution time.
Digital twin	Verification, timing estimation, parameter calibration and regression testing.	Hardware-in-the-loop tests.	Simulation supports assurance but does not replace formal planning.
Code generation	Sandboxed adapter/executor synthesis with gated release.	Manual implementation.	Captures Code-as-Policy benefits without granting uncontrolled execution authority.

7.2 Planner-selection logic

7.3 Research hypotheses

7.4 Fallback strategy

If automatic domain generation remains below acceptance thresholds, the project will retain a manually approved domain library and use LLMs only for problem generation and dialogue. If code generation does not pass the assurance gate reliably, missing skills will be implemented manually while the system still reports capability gaps. If PlanSys2 cannot support a required PDDL feature, the planning core will remain behind the Unified Planning abstraction and a custom executor adapter will translate the validated plan to MI.RA/Dexter/BehaviorTree.CPP.

Table 14 Planner-selection logic by problem feature.

Problem feature	Preferred method
Discrete deterministic actions, no durations	Classical PDDL planner.
Durative actions, concurrency, preferences	Temporal PDDL planner such as POPF/OPTIC where supported.
Heterogeneous agents, resource capacities, calendars, assignment	Constraint programming/CP-SAT, potentially using an LLM-generated task skeleton.
Geometric feasibility and manipulation	Task-and-motion planning/MoveIt 2 feasibility service.
Uncontrollable action durations	Temporal plan plus STNU controllability/dispatch analysis.
Stochastic outcomes or partially observable state	Research branch using contingent/stochastic planning; do not force into deterministic PDDL.
Emergency runtime deviation	Approved recovery BT or bounded plan repair; safe stop if validation deadline cannot be met.

Table 15 Research hypotheses and their evaluation.

ID	Hypothesis	Evaluation
H1	Evidence-fused capability discovery produces more executable planning domains than text-only domain generation.	Compare domain validity and action-grounding success across discovery ablations.
H2	A typed IR plus deterministic compiler and validator loop outperforms direct PDDL generation for compact local models.	Measure valid-domain/problem rate, repair iterations, latency and model size.
H3	Hybrid PDDL + CP scheduling improves feasibility and makespan for multi-agent industrial tasks compared with LLM-only and PDDL-only baselines.	Benchmark increasing resources, precedence, calendars and agent heterogeneity.
H4	Scene-graph and capability-registry grounding reduces hallucinated objects/actions and improves replanning accuracy.	Compare raw visual/text context, scene graph only and combined operational graph.
H5	Generated adapters can reduce integration effort without increasing operational risk when passed through the assurance gate.	Measure engineering time, defects, test coverage and blocked unsafe releases.
H6	A model-agnostic architecture allows smaller local models to meet industrial requirements for most structured tasks.	Compare local and hosted models under fixed validators and the same benchmark.

8 Benchmarking and validation plan

8.1 Benchmark philosophy

Evaluation must cover the complete no-code chain. A system is not successful because it produces plausible text or valid PDDL syntax. It succeeds when the user's intent is represented correctly, the model is valid, the solver finds a feasible/optimal plan, each action is grounded, the plan executes under controlled perturbations and the process is faster for a process expert than the baseline programming method.

8.2 Benchmark domains

Table 16 Benchmark domains.

Domain	Core capabilities	Planning challenges
Single-arm pick and place	Object detection, grasp, move, place, I/O.	Grounding, missing objects, parameter binding and simple ordering.
Quality inspection	Camera setup, acquire, inspect, classify, report.	Conditional effects, information gathering and uncertain outcomes.
Welding process	Tool setup, seam selection, welding, inspection.	Durations, process constraints, no mid-seam replanning and safety stops.
Mechanical disassembly/remufacturing	Fastener/tool identification, grasping, removal and part handling.	Scene-graph precedence, accessibility, tool resources and effect validation.
Multi-robot material handling	Multiple arms/mobile manipulators and shared zones.	Allocation, scheduling, resource conflicts, travel time and makespan.
Human-robot collaborative assembly	Human and robot skills, shared resources and safety zones.	Uncertain human durations, task allocation, safe coordination and replanning.

8.3 Dataset structure

- At least 10 cell/domain contexts for capability and domain discovery, consistent with the original KB KPI.
- At least 100 natural-language task requests for problem generation, with paraphrases, ambiguity and missing information.
- At least 100 plan-generation cases spanning classical, temporal and resource-constrained problems.
- At least 100 plan-to-execution cases with ground-truth skill mappings and parameter bindings.
- Perturbation variants: missing node, unavailable tool, delayed action, changed object state, failed perception, resource conflict and operator intervention.
- Versioned benchmark artefacts: registry snapshots, domains, problems, plans, validation logs, expected execution traces and expert judgements.

8.4 Metrics and proposed thresholds

Table 17 Metrics and proposed acceptance thresholds.

Metric	Definition	Proposed acceptance/target
Capability discovery precision/recall	Correct discovered capabilities and required fields against expert registry.	Report per source and fused result; target ≥ 0.90 F1 for approved capability classes.
Provenance completeness	Share of registry facts linked to source and confidence.	100% for facts used in released planning models.

Metric	Definition	Proposed acceptance/target
Domain validity	Syntactic/semantic validation and ability to solve reference instances.	100% syntax; $\geq 90\%$ semantically correct before human repair; 100% for released domains.
Problem validity	Typed objects, initial state and goal consistent with domain.	$\geq 95\%$ after automatic repair; 100% before planning release.
Plan validity	Independent validator accepts the plan.	100% of executed plans.
Goal correctness	Plan achieves the interpreted user goal and constraints.	$\geq 95\%$ on benchmark; disagreements adjudicated by experts.
Grounding success	Actions mapped to approved executors with valid parameters.	$\geq 90\%$ without new code; 100% before execution.
Optimality/schedule quality	Makespan/cost relative to optimum or best known baseline.	Report gap; no regression against selected solver baseline.
Programming-time reduction	Time from request to approved executable workflow versus current MI.RA/Dexter/manual process.	Maintain contractual 66% target.
Non-expert usability	Task completion by process expert without robotics programming training.	Maintain contractual objective; measure success, workload and interventions.
Runtime robustness	Successful completion under defined perturbations.	Domain-specific threshold; zero unsafe continuation after invalid state.
Generated-code release rate	Candidate code passing all gates without manual correction.	Research metric; no minimum, zero bypasses of assurance gate.
Latency and compute	End-to-end and stage-level time, memory, GPU/CPU and model calls.	Meet task-specific interactive and replanning budgets.
Fleet scale	Supported agents and concurrent tasks.	3 mobile manipulators or 4 industrial arms, consistent with baseline.

8.5 Baselines and ablations

- Direct LLM plan generation.
- Direct natural-language-to-PDDL generation.
- Text-only domain generation versus ROS 2-only, documentation-only, scene-only and fused discovery.
- Single planner versus planner portfolio and PDDL-only versus PDDL+CP.
- Scene graph versus raw perception summary versus combined operational knowledge graph.
- Hosted frontier model versus local open-weight model under identical validators.
- With and without human confirmation at ambiguity gates.
- With and without digital-twin verification.

8.6 Statistical treatment

Report success proportions with confidence intervals, paired comparisons on the same tasks, medians and percentile latency, and bootstrap intervals for optimality/time differences. Separate model-generation variance from planner determinism by repeating probabilistic stages with fixed and varied seeds. Publish the exact model identifiers, prompts, schemas, tool versions and artefact hashes for each benchmark run.

8.7 Mockup evaluation and evidence for the journal manuscript

The initial evaluation follows [1, Sec. 3], starting with a simplified single-robot pick-and-place scenario. It will assess problem-generation validity, repair success, planning success, plan quality and execution-grounding success. Execution completion and observed effects will be recorded separately from planning success. Failure cases will include missing information, unavailable skills and unsuccessful grounding where supported by the implemented mockup.

The proposal reports preliminary use of Llama 3.1–8B up to symbolic plan generation and plans comparisons between open-weight and closed-source models. This revision does not introduce numerical results or claim a completed comparison. The manuscript will report the exact tested backends, versions, prompts, planner configuration, cases and computational environment, with repeated runs where needed to characterise model variability.

The manuscript evidence package will include the architecture, implemented scope, reproducible protocol, measured results, failure analysis and limitations. Initial prototype findings will be distinguished from the later A2-wide KPI campaign, including the 66% programming-time reduction and target fleet scale. Journal submission will provide an additional dissemination output; the running mockup and its evidence remain part of the M12 D2.1 delivery.

9 COMAU contributions to industrial design

This section structures the evidence for COMAU participation in the first-year design of Activity 2. The technical specification assigns the Host Institution a role in use-case development, interaction with the Research Organization and sharing simulation environments and tools. The material currently available does not establish which of the contributions below were performed, by whom or for how long. Each entry therefore remains subject to confirmation through project records.

9.1 Reporting scope and evidence status

Separate three categories: pre-existing technology and know-how; project-specific activities performed during the reporting period; and work planned for later phases. Record an activity as completed only when its contributor, period, output and supporting evidence can be identified. A design input may have substantial scientific value without implying a large volume of personnel effort. Financial effort is not estimated from this technical contribution matrix.

9.2 Potential contributions and expected evidence

The following matrix is a framework for collecting and assessing contributions. It is not a statement of completed work.

Table 18 Potential COMAU contributions and expected evidence.

Potential COMAU contribution	Effect on method and mockup	Evidence to retrieve
Dexter programming workflow analysis	Identify decisions that currently require specialist expertise and those suitable for automation.	Annotated demonstrations, example workflows, meeting notes and emails.
Block-library analysis	Specify how a block becomes a plannable skill: parameters, preconditions, effects, resources and errors.	Module descriptions, configuration examples and technical annotations.
Industrial requirements definition	Inform structured generation, validation and explicit handling of missing capabilities.	Requirements, architecture review comments and documented decisions.
Use-case definition	Define realistic process sequences, constraints, variants and mockup failure conditions.	Use-case specification, process examples and test scenarios.
Future interface design	Specify the mapping between symbolic actions, ROS 2 skills and Dexter modules for subsequent integration.	Interface diagrams, preliminary mappings and compatibility assessments.
Mockup review and acceptance criteria	Assess industrial relevance and define how the usefulness of the prototype will be measured.	Demo feedback, annotated tests, issue lists and resulting corrections.

9.3 Contribution records

Create one record per contribution confirmed by COMAU, using the fields below. Retain unresolved records as pending. A note prepared after the activity should state its preparation date and identify the earlier records used to reconstruct the contribution.

9.4 Design decisions to document

Capability semantics. Identify whether COMAU input defined module preconditions, effects, parameter constraints, failure conditions or success observations beyond the existing block interface. Link each confirmed input

Table 19 Fields of a contribution record.

Field	Information to complete
Record identifier and status	COMAU-A2-[number]; pending evidence, confirmed, or planned.
Contributor and period	Named contributor, team, date or reporting period, and role.
Input and source	Requirement, workflow, module description, review comment or simulation asset; exact source reference and date.
Design decision and attribution	Decision informed by the input, alternatives considered, and distinction between COMAU input, joint decisions and Research Organization development.
Mockup or method effect	Affected component, interface, experiment, code version or document section.
Verification and follow-up	Review result, unresolved issue, responsible person and completion status.
Effort record reference	Reference to applicable personnel records, if available. Do not infer hours from the contribution category or budget.

to the corresponding capability record or formal action model.

Use case and evaluation. Identify whether COMAU supplied process sequences, realistic variants, operating constraints, baseline workflows or acceptance criteria. Link confirmed contributions to the M12 demonstration and the evaluation protocol in Section 8.7.

Architecture and integration preparation. Identify whether COMAU reviewed the separation between planning, validation and execution, the mapping to Dexter modules, or compatibility with simulation and deployment environments. Record future integration actions separately from completed first-year design work.

9.5 Contribution summary for the reporting period

To be completed after the contribution records have been reviewed: confirmed COMAU activities and outputs; their effect on research directions and mockup design; contribution boundaries relative to the Research Organization; outstanding evidence; and actions planned for the next period. The final summary should cite the record identifiers in Section 9.3 and include only substantiated activities.

10 Updated implementation roadmap

10.1 M12 mockup and work breakdown M13–M36

Table 20 M12 mockup and work breakdown M13–M36 with decision gates.

Period	Technical focus	Main outputs/decision gate
M12	First running mockup based on the neuro-symbolic workflow in [1].	D2.1 scientific plan, running mockup, demonstration and reproducibility evidence; Gate D0.
M13–M15	Benchmark formalisation; capability ontology; discovery adapters for ROS 2 and MI.RA/Dexter.	Registry schema v1; benchmark v1; decision on supported PDDL subset and planning API.
M16–M18	Multi-source capability extraction; typed IR; deterministic PDDL compiler; validators.	Discovery MVP; NL-to-IR-to-PDDL prototype; Gate D1 on domain/problem validity.
M18–M21	Planner portfolio, CP scheduling and action-grounding service.	Validated plan pipeline; skill matching; Gate D2 on plan validity and grounding.
M20–M24	PlanSys2/BehaviorTree.CPP execution; temporal network and uncertain-duration monitoring.	Execution prototype; plan-to-BT mapping; Gate D3 on temporal and runtime behaviour.
M22–M27	Digital-twin verification, timing/cost learning and selected TAMP integration.	Simulation verification pipeline; schedule-quality evidence; MCTS go/no-go for selected domains.
M25–M30	Code-generation assurance workflow and MI.RA/Dexter integration.	Sandbox, tests, signed containers; integrated low-code workflow; Gate D4 on software assurance.
M28–M34	Online state/effect monitoring, plan repair, replanning and approved recoveries.	Runtime monitoring and recovery library; Gate D5 on robustness and safety response.
M31–M36	Industrialisation, fleet-scale evaluation, documentation and handover.	Benchmark report, KPI evidence, integration package and final Activity 2 technical baseline.

10.2 Mapping to contractual tasks

Table 21 Updated internal interpretation of the contractual tasks.

Contractual task	Updated internal interpretation
T2.2	Capability and domain discovery; natural-language problem generation; typed IR; lightweight/local model benchmarking; deterministic PDDL compilation.
T2.3	Validated temporal plan-to-execution transformation; BT generation from selected plans; STN/STNU dispatch and duration-uncertainty monitoring.
T2.4	Hybrid planner/CP optimisation; digital-twin verification and timing/cost estimation; optional MCTS/TAMP research branches.
T2.5	Grounded plan execution; expected-effect monitoring; bounded plan repair/replanning; approved recovery and safe fallback.
T2.6	MI.RA/Dexter block discovery, workflow generation, containerised integration and industrial security assurance.
T2.7	CI/CD, model and prompt versioning, test automation, SBOM, signing, reproducible benchmark and audit logs.
T2.8	Benchmark governance, ground truth, end-user tests and ongoing regression evaluation.

10.3 Responsibilities

Table 22 Responsibilities per role/organisation.

Role/organisation	Primary contribution
PI/Activity lead	Scientific direction, decision gates, integration priorities, and acceptance of methodological changes.
CNR	Architecture, capability discovery, LLM workflows, robotics integration, code assurance, benchmarks, and prototypes.
CNR/planning specialists	Planning/scheduling formalisms, planner portfolio, temporal uncertainty, validation, and optimization methods.
COMAU/Host Institution	Industrial requirements, use-case definition, Dexter capability/interface input and mockup review; later integration and end-user evaluation. First-year contributions are to be documented in Section 9.
Activity 3/4 teams	Scene/state interfaces, motion-feasibility services, safety-aware motion planning, and execution feedback.
Activity 6 team	Human oversight, usability, trust, workload and acceptance measures.

10.4 Decision gates

- **D0 (M12):** deliver the first running mockup with the scientific plan, demonstration instructions, versioned artefacts and observed execution evidence. Record implemented functions and limitations separately from later research extensions.
- **D1 (M18):** proceed with automatic domain generation only if released domains achieve 100% formal validation and acceptable semantic correctness; otherwise, use approved domain templates.
- **D2 (M21):** select PDDL-only or PDDL+CP architecture based on multi-agent benchmark feasibility and schedule quality.
- **D3 (M24):** confirm PlanSys2/BT execution route and supported temporal-uncertainty method.
- **D4 (M30):** permit generated adapters in integration only after zero gate bypasses and successful rollback tests.
- **D5 (M34):** approve online replanning only for scenarios where state updates, validation and safety response meet deadline budgets.

10.5 Scientific manuscript and submission target

A manuscript describing the first running mockup is planned for submission within one month of the 23 September 2026 revision, by 23 October 2026. The candidate destinations are IEEE Transactions on Automation Science and Engineering (IEEE T-ASE) and IEEE Transactions on Industrial Informatics (IEEE TII); the team will select one venue for submission.

The manuscript will present the neuro-symbolic architecture, the implemented mockup, the initial evaluation protocol and measured experimental results. Preparation comprises consolidating the mockup and reproducibility materials, completing experiments and failure analysis, finalising the manuscript and obtaining co-author approval. These steps support the stated submission target and do not imply that the experiments or submission have already occurred.

At submission, the project will record the selected journal, manuscript title, author list, submission date and acknowledgement or manuscript identifier. Acceptance and publication will be recorded separately if they occur. The submission date is a calendar target; no equivalence with the project M12 date is assumed in this document.

11 Risks, ethics and regulatory alignment

11.1 Technical and operational risk register

Table 23 Technical and operational risk register (likelihood/impact: L = low, M = medium, H = high).

Risk	Likelihood/impact	Mitigation	Indicator
Hallucinated capability or action semantics	M/H	Multi-source evidence, provenance, schema constraints, expert approval and executable tests.	Unsupported symbols; grounding failures; provenance gaps.
Incorrect PDDL domain/problem	M/H	Typed IR, deterministic compiler, VAL/UP checks, reference problems and repair loop.	Validation failures; unsolved reference instances.
Valid but non-executable plan	M/H	First-class grounding map, resource and parameter checks, digital-twin execution.	Ungrounded actions; runtime precondition failures.
Unsafe or malicious user instruction	M/H	Policy gate, role-based access, safety constraints, prompt sanity checks and human approval.	Rejected requests; attempted policy bypass.
Prompt injection through documents/network metadata	M/H	Treat retrieved content as data, source allow-list, instruction isolation and tool permission boundaries.	Unexpected tool calls; retrieval-policy violations.
Generated insecure code	M/H	Restricted targets, static/security analysis, dependency allow-list, tests, review, signing and rollback.	Gate failures; vulnerabilities; unsigned artefacts.
Model/provider drift	H/M	Provider abstraction, frozen benchmark, model cards, requalification and local fallback.	Regression at decision gate; changed API behaviour.
Planner/representation mismatch	M/M	Feature analyser, planner portfolio, explicit unsupported-feature errors.	Engine failures; semantic translation gaps.
Temporal uncertainty exceeds model	M/H	Observed duration distributions, STNU/robust margins, replanning and safe stop.	Missed windows; non-controllability.
Discovery exposes sensitive network data	M/H	Least privilege, no passive sniffing by default, network segmentation, data minimisation and logs.	Unauthorised endpoints; excessive data collection.
Over-reliance on simulation	M/M	Hardware-in-the-loop, staged commissioning and real execution evidence.	Simulation-to-real discrepancy.
Insufficient non-expert usability	M/M	Mixed-initiative dialogue, explanation, user-centred testing and interface simplification.	High intervention/workload; low task completion.

11.2 Human oversight and explainability

The system should expose a concise decision record: interpreted goal; assumptions; capabilities selected; unavailable or inferred information; formal constraints; objective function; selected plan and alternatives; validation results; generated-code status; and reasons for replanning or refusal. Human confirmation is mandatory for new domain approval, new code release, safety-relevant ambiguity and changes that exceed the validated operating envelope.

11.3 EU AI Act

Regulation (EU) 2024/1689 establishes harmonised rules for AI systems [41]. The exact classification of each **Plan4ARI** component depends on its intended purpose and integration into machinery. The project should nevertheless implement risk management, data and technical documentation, logging, transparency, human oversight, accuracy, robustness and cybersecurity by design. The D2.1 architecture supports these requirements through traceable evidence, independent validation, access control and explicit approval gates.

11.4 Machinery and cybersecurity regulation

Regulation (EU) 2023/1230 on machinery becomes the relevant machinery framework from January 2027, while Regulation (EU) 2024/2847 introduces horizontal cybersecurity requirements for products with digital elements [42, 43]. **Plan4ARI** should therefore maintain a clear boundary between experimental AI modules and safety functions, protect update and deployment channels, manage vulnerabilities and dependencies, preserve technical documentation and support secure rollback. This deliverable is not a conformity assessment; legal classification and harmonised-standard mapping require specialist review.

11.5 Data protection and confidentiality

- Minimise collection of user speech, network metadata and production documents.
- Define retention, access and lawful basis for personal data used in dialogue and evaluation.
- Keep proprietary process data on-premise when required; use local models or approved enterprise endpoints.
- Redact or isolate credentials, personal data and trade secrets from model context and logs.
- Document training/fine-tuning datasets and prevent unauthorised reuse of customer data.

11.6 Research ethics

Experiments involving workers must use informed consent, appropriate ethics review, data minimisation and separation between research evaluation and employment assessment. The no-code interface must not create a misleading impression that the robot understands more than it does. Uncertainty, assumptions, unavailable capabilities and reasons for refusal should be communicated clearly.

12 Conclusions and proposed updates

12.1 Final scientific position

The original **Plan4ARI** objective remains timely. The most credible 2026 research position is not to make an LLM the planner, but to make it the semantic and knowledge-engineering layer of a verified planning system. Formal solvers, validators, explicit capability models and trusted executors provide determinism and industrial control. The resulting architecture is neuro-symbolic, capability-aware and model agnostic.

The restructuring proposal is therefore recommended, with two refinements. First, capability discovery must be broader and more controlled than ROS 2 network sniffing: it requires semantic fusion across runtime interfaces, engineering data, documentation, code and scene state. Second, Code as Policies should be adopted as an inspiration for composing APIs and generating adapters, not as permission for unrestricted policy code to control industrial equipment.

12.2 Proposed approval decisions

1. Approve operational capability discovery and skill registry as the central research contribution of Activity 2.
2. Approve the NL-to-IR-to-PDDL formalization pipeline and independent validator/solver repair loop.
3. Approve a hybrid planner portfolio including classical/temporal planning and constraint programming.
4. Approve PlanSys2 and BehaviorTree.CPP as reference open execution technologies, subject to feature and integration validation.
5. Approve STN/STNU methods as temporal-dispatch and uncertainty-monitoring components.
6. Approve digital-twin verification and timing/cost estimation; retain MCTS as a domain-dependent research option rather than the baseline optimiser.
7. Approve gated code generation for symbolic solvers, adapters and candidate action executors; prohibit direct deployment or safety-critical generation.
8. Approve the revised benchmark and KPI framework while preserving the contractual high-level targets.
9. Approve the M12 mockup evidence package and the updated M13–M36 development roadmap and decision gates.
10. Place model families, VLA/world-action models and fully automatic domain generation on a monitored technology watch list.
11. Confirm the manuscript submission target of 23 October 2026 to one selected journal, IEEE T-ASE or IEEE TII, while recording acceptance and publication separately.

12.3 Open research questions

- How much semantic capability information can be inferred reliably from a running ROS 2 graph without manually authored annotations?
- How should conflicting evidence from code, documentation, runtime behaviour and vendor metadata be reconciled?
- Which skill ontology provides sufficient expressiveness without making domain generation too complex?
- When should scheduling be compiled into PDDL, solved in CP, or handled by a coupled planner?
- Can operational duration distributions be learned online while preserving temporal controllability and safety?
- What is the minimum assurance evidence required for generated adapters in an industrial low-code platform?
- How do scene-graph uncertainty and capability uncertainty propagate into plan selection and explanations?
- How should user preferences and trust measures influence optimisation without weakening safety constraints?

References

- [1] S. Sandrini, “Neuro-symbolic planning and scene-graph-based reasoning for no-code robotic task programming,” Research proposal draft (A2 Research Project Proposal), Jul. 2026, 9 July 2026.
- [2] Plan4ARI, “Planning for artificial robotic intelligence and beyond, project description v3/definitive proposal,” especially Sections A.2–A.3 and Activity 2 work plan.
- [3] W. Zhai, J. Liao, Z. Chen, B. Su, and X. Zhao, “A survey of task planning with large language models,” *Intelligent Computing*, 2025.
- [4] H. Wei *et al.*, “A modern survey of LLM planning capabilities,” in *Proceedings of ACL 2025*, 2025, aCL Anthology: 2025.acl-long.958.
- [5] M. Tantakoun, X. Zhu, and C. Muise, “LLMs as planning formalizers: A survey for leveraging large language models to construct automated planning models,” in *Findings of ACL 2025*, 2025, aCL Anthology: 2025.findings-acl.1291.
- [6] M. Chiari, L. Putelli, N. Rossetti, I. Serina, and A. Gerevini, “On planning through LLMs,” in *Proceedings of ICAPS 2025*, 2025.
- [7] S. Kambhampati *et al.*, “LLMs can’t plan, but can help planning in LLM-modulo frameworks,” in *Proceedings of ICML 2024*, ser. PMLR, vol. 235, 2024, pp. 22 895–22 907.
- [8] K. Valmeekam, K. Stechly, and S. Kambhampati, “LLMs still can’t plan; can LRMs? A preliminary evaluation on PlanBench,” arXiv:2409.13373, 2024.
- [9] B. Liu *et al.*, “LLM+P: Empowering large language models with optimal planning proficiency,” arXiv:2304.11477, 2023.
- [10] E. Gestrin, M. Kuhlmann, and J. Seipp, “NL2Plan: Robust LLM-driven planning from minimal text descriptions,” arXiv:2405.04215, 2024/2025.
- [11] J. Oswald, K. Srinivas, H. Kokel, J. Lee, M. Katz, and S. Sohrabi, “Large language models as planning domain generators,” in *Proceedings of AAAI 2024*, 2024, arXiv:2405.06650.
- [12] P. P. Kagitha, A. Zhu, and L. Zhang, “Addressing the challenges of planning language generation,” arXiv:2505.14763, 2025.
- [13] Y. Hao, Y. Chen, Y. Zhang, and C. Fan, “Large language models can solve real-world planning rigorously with formal verification tools,” arXiv:2404.11891, 2024.
- [14] J. Palmieri, M. Lippi, and A. Marino, “Hybrid task planning and scheduling in heterogeneous multi-agent systems based on LLMs and constraint programming,” *Robotics and Autonomous Systems*, vol. 202, p. 105480, 2026.
- [15] Y. Chen, Y. Hao, Y. Zhang, and C. Fan, “Code-as-symbolic-planner: Foundation model-based robot planning via symbolic code generation,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025.
- [16] B. Schenkenfelder *et al.*, “Leveraging the power of LLMs to transform robot programs into low-code,” in *IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2024, pp. 1–4.
- [17] M. Henzel, M. Böggemann, E. Witte, and F. Daut, “Enabling robot-assisted production through natural language input,” in *Experiment@ International Conference 2025*, 2025, pp. 121–125.
- [18] D. Syniawa, L. Droste, and B. Kuhlenkötter, “Semi-automated programming of industrial robotic systems using large language models and standardized data model,” *Robotics*, vol. 15, no. 4, p. 79, 2026.
- [19] C. E. Mower *et al.*, “ROS-LLM: A ROS framework for embodied AI with task feedback and structured reasoning,” arXiv:2406.19741, 2024.

- [20] J. Liang *et al.*, “Code as policies: Language model programs for embodied control,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [21] M. Fox and D. Long, “PDDL2.1: An extension to PDDL for expressing temporal planning domains,” *Journal of Artificial Intelligence Research*, vol. 20, pp. 61–124, 2003.
- [22] Plan4ARI, “Deliverable D2.1 – draft outline, updated scientific plan for low-code programming with generative and deliberative AI,” Jul. 2026, 20 July 2026.
- [23] A. Micheli *et al.*, “Unified planning: Modeling, manipulating and solving AI planning problems in Python,” *SoftwareX*, vol. 29, p. 102012, 2025.
- [24] R. Howey, D. Long, and M. Fox, “VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL,” in *IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, 2004.
- [25] S. Panjkovic and A. Micheli, “Abstract action scheduling for optimal temporal planning via OMT,” in *Proceedings of AAAI*, vol. 38, 2024, pp. 20 222–20 229.
- [26] F. Martín, J. Ginés, V. Matellán, and F. J. Rodríguez, “PlanSys2: A planning system framework for ROS2,” arXiv:2107.00376, 2021.
- [27] M. Mayr, F. Rovida, and V. Krueger, “SkiROS2: A skill-based robot control platform for ROS,” arXiv:2306.17030, 2023.
- [28] M. Pantano, T. Eiband, and D. Lee, “Capability-based frameworks for industrial robot skills: A survey,” in *IEEE International Conference on Automation Science and Engineering (CASE)*, 2022.
- [29] C. Zhang Sprenger *et al.*, “ORPP—an ontology for skill-based robotic process planning,” *Electronics*, vol. 13, no. 18, p. 3666, 2024.
- [30] L. M. V. da Silva *et al.*, “Toward a method to generate capability ontologies from natural language using large language models,” arXiv:2406.07962, 2024.
- [31] T. Westermann, M. Ramonat, J. Hujer, F. Gehlhoff, and A. Fay, “Automatic mapping of AutomationML files to ontologies for graph queries and validation,” arXiv:2504.21694, 2025.
- [32] S. Profanter, A. Breitzkreuz, M. Rickert, and A. Knoll, “A hardware-agnostic OPC UA skill model for robot manipulators and tools,” in *IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2019.
- [33] L. M. V. da Silva, A. Köcher, M. S. Gill, M. Weiss, and A. Fay, “Toward a mapping of capability and skill models using asset administration shells and ontologies,” arXiv:2307.00827, 2023, revised 2024.
- [34] A. Cimatti, M. Do, A. Micheli, M. Roveri, and D. E. Smith, “Strong temporal planning with uncontrollable durations,” *Artificial Intelligence*, vol. 256, pp. 1–34, 2018.
- [35] K. Rana *et al.*, “SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning,” in *Conference on Robot Learning*, ser. PMLR, vol. 229, 2023.
- [36] J. Herzog, J. Liu, and Y. Wang, “Domain-conditioned scene graphs for state-grounded task planning,” arXiv:2504.06661, 2025.
- [37] Z. Yang, C. Garrett, D. Fox, T. Lozano-Pérez, and L. P. Kaelbling, “Guiding long-horizon task and motion planning with vision language models,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, arXiv:2410.02193.
- [38] S. Li *et al.*, “Safe planner: Empowering safety awareness in large pre-trained models for robot task planning,” in *Proceedings of AAAI*, vol. 39, no. 14, 2025, pp. 14 619–14 627.
- [39] Y. Wu *et al.*, “SELP: Generating safe and efficient task plans for robot agents with large language models,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

- [40] I. Obi *et al.*, “SafePlan: Leveraging formal logic and chain-of-thought reasoning for enhanced safety in LLM-based robotic task planning,” arXiv:2503.06892, 2025.
- [41] European Union, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI act),” Official Journal of the European Union, 2024.
- [42] —, “Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (cyber resilience act),” 2024.
- [43] —, “Regulation (EU) 2023/1230 on machinery,” 2023.
- [44] I. Singh *et al.*, “ProgPrompt: Generating situated robot task plans using large language models,” *Autonomous Robots*, vol. 47, pp. 999–1012, 2023.
- [45] M. Ahn *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Conference on Robot Learning*, ser. PMLR, vol. 205, 2023.
- [46] K. Lin *et al.*, “Text2Motion: From natural language instructions to feasible plans,” *Autonomous Robots*, vol. 47, pp. 1345–1365, 2023.
- [47] R. A. Izzo, G. Bardaro, and M. Matteucci, “BTGenBot: Behavior tree generation for robotic tasks with lightweight LLMs,” arXiv:2403.12761, 2024.
- [48] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018.
- [49] BehaviorTree.CPP, “BehaviorTree.CPP and BehaviorTree.ROS2 official documentation and repositories,” accessed 21 July 2026.